

Final Project - Urban Poverty Mapping from Satellite Imagery

Deep Learning for Social Sciences 2025

1 Introduction

Even though poverty is globally declining, there were still 713 million people living in poverty in 2022 [1]. Fighting poverty therefore remains one of the biggest challenges of our time. With the rise of machine learning, the question is: how can we use these new tools to help reduce poverty globally?

One of the main problems in the fight against poverty is that there is not a lot of regional data available. We realized this quickly when we started working on our project. While there is plenty of data at the country level, the amount becomes less and less when looking at smaller regions or cities. The reason is often that such data is based on household surveys. At small scales, there are data protection issues, which is why this kind of information is usually not available. Furthermore, collecting this data is very difficult, especially in poorer regions. Poverty and illiteracy often correlate [2], which means you cannot simply conduct online surveys but actually have to talk to people directly.

The rise of deep learning techniques has opened up new opportunities to address problems like this. One of these techniques is the architecture of Convolutional Neural Networks (CNNs), models that learn features through filter optimization when fed with images.

The idea behind this project is to show that CNN architectures can – to some extent – support poverty mapping by being trained with satellite images combined with existing poverty data for the same geographical locations. The assumption is that non-poor regions may show different infrastructure, vegetation, or housing patterns compared to poorer regions. The advantage of satellite images is their wide coverage, frequent updates, and - at least in academic contexts through Google Earth Engine - free accessibility. This project demonstrates that a CNN can learn such structural patterns to some extent and classify or predict poverty levels accordingly.

But in order to build such a CNN, we first needed to find suitable data to work with. In Section 2, we describe our search process and explain how we ended up using poverty data from Nigeria provided by the WorldPop dataset. Using longitude and latitude information, we matched poverty scores to satellite images and categorized them into four classes: non-poor, vulnerable, poor, and severely poor. After collecting and preprocessing the data, we created training, validation, and test splits. We then trained and evaluated four different models: a baseline CNN with our own architecture, as well as a ResNet50 architecture in three variations (pretrained, fine-tuned in One-Step, and fine-tuned in Two-Steps). We compared the results, tested the generalizability on data from Kenya, and also extended our analysis to a regression approach. Finally, we discuss the implications of our findings for policy-making and provide a conclusion with directions for future research.

Ultimately, our results illustrate that combining modern deep learning methods with open satellite data can help AI play a role in addressing one of the most pressing global challenges.

2 Data Collection

To start off, we thought about how to obtain data that we could use to train the CNN models. Since we knew that we could get a 1 km² patch of satellite images using longitude and latitude data, we searched for regional poverty data that also divided a country into 1 km² patches with longitude and latitude information. This way, we wanted to match the two datasets through their coordinates. After some research, we found the WorldPop dataset [3] provided by the World Bank. It includes longitude and latitude coordinates of each patch as well as a poverty score. The poverty metric ranges from 0 to 1 and indicates the percentage of people in poverty per grid square.

Several countries were available in WorldPop, but we decided to use the Nigeria dataset from 2010. Nigeria is a good choice because it has varied land cover - in the north it is mostly desert-like, while other regions have vegetation and coastal zones. In addition, Nigeria has large cities distributed across the country (see Figure 1). Another reason is that Nigeria is a priority country for international development programs, so a policy impact analysis makes sense. For Nigeria, the poverty score was based on a monetary threshold approach: extreme poverty was defined as living on at most \$1.25 per day (2011 Purchasing Power Parity).



Figure 1: Satellite Image of Nigeria.

Later on, as mentioned in the introduction, we tested our model on Kenya to check for generalizability. Kenya also had a WorldPop dataset structured similarly to Nigeria’s. At first, we considered using a dataset from Asia, but we decided the structural differences would be too large and instead focused on generalization within one continent. We decided to use Kenya because, while it is comparable to Nigeria in that both have dense cities as well as rural areas, the landscapes are quite different. In contrast to Nigeria, Kenya’s geography includes savannahs and mountains. This made Kenya different enough to be a good generalization test. The Kenya poverty scores were based on the Multidimensional Poverty Index (MPI), which measures acute deprivations in health, education, and living standards [4].

After identifying the datasets, we had to answer two questions. First: how to define a threshold to determine whether a region is urban or not. We wanted to focus on urban areas because more people live in them, so poverty mapping there directly affects more lives. From a practical perspective, limiting the dataset to urban areas also helped keep the dataset size manageable. The second question was how to define thresholds for classes, since our first approach was a multi-class classification task.

After research and testing different thresholds from various sources, we decided to define “urban” as patches with more than 800 people per grid [5]. This gave us around 36,000 data points, which is a reasonable amount for training a CNN. For poverty classes, we used thresholds from the Human Development Reports [6] and categorized into four classes according to their poverty thresholds: those regions with a share of less than one fifth (0.2) were classified as non-poor; those between one fifth (0.2) and one third (0.33) as vulnerable to poverty; those between one third (0.33) and one half (0.5) as poor; and those with at least one half (0.5) as severely poor. From a coding perspective, we loaded the WorldPop CSV file, created a dataframe, applied our thresholds, and exported the processed data into a new CSV file. This new file was then used for the collection of satellite images.

For the imagery, we used the harmonized Sentinel-2 dataset from Google Earth Engine [7], which provides atmospherically corrected Sentinel-2 surface reflectance imagery at 10 m resolution. We selected a wide date range - from the start of Sentinel-2 image collection (23.06.2015) until today (meaning the 22nd of August 2025) - since the dataset has frequent revisit times and harmonizes across sensors for consistency. This ensured fewer problems with cloud coverage or noise.

For the collection process, we loaded the CSV with the poverty data, retrieved the corresponding satellite images using longitude and latitude (converting to UTM zones to ensure correct patching), and stored them locally. The code was written so that each image was automatically placed in the correct folder based on the poverty class. We also created a metadata CSV file, which contained among other things image name and poverty score to enable the later regression analysis.

In the end, we had four folders of satellite images (one for each poverty class) as well as the metadata file.

3 Classification

3.1 Preliminary Analysis

To start off our preliminary analysis, we visualized the poverty data from the WorldPop dataset to get an idea of how poverty is distributed across Nigeria. Figure 2a shows clear regional differences: while the proportion of people living in poverty is relatively high in the north, it is somewhat lower in the south. This matches

the geographical conditions - the north is drier and more desert-like, while the south has more vegetation and coastal access, which may partly explain the lower poverty levels there. Overall, Nigeria appears to be a relatively poor country, with on average a higher proportion of people living in poverty.

Next, we checked which regions would be included under our urban threshold. As shown in Figure 2b, there is good spatial coverage, so data from across the country was used. The density map also shows that the south is more populated, which aligns with both the poverty map (Figure 2a) and the general geographical differences across regions (Figure 1).

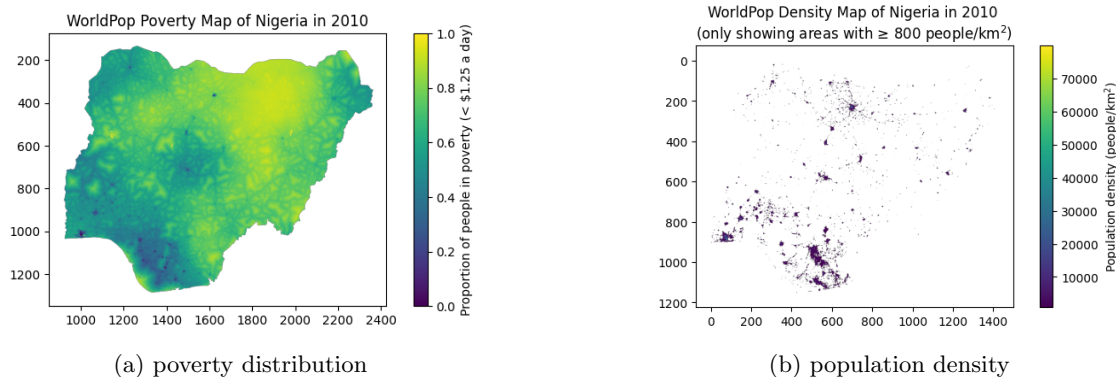


Figure 2: WorldPop maps of Nigeria in 2010.

In total, our dataset consisted of 36,417 images. The class distribution (Figure 3) shows a strong imbalance: most images belong to the severely poor class (19,421), while only 753 are labeled as non-poor, making up just about 2% of the data. Furthermore, 2,742 images are in the vulnerable class and 13,501 in the poor class. We considered trying alternative thresholds, but since the ones we chose are standard and reflect reality, we decided not to artificially rebalance. Instead, we kept the imbalance in mind and controlled for it later in model training, as otherwise the CNN might have been biased towards predicting severely poor more often.

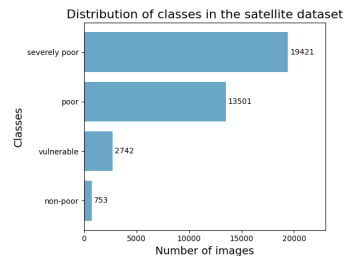


Figure 3: Class Distribution.

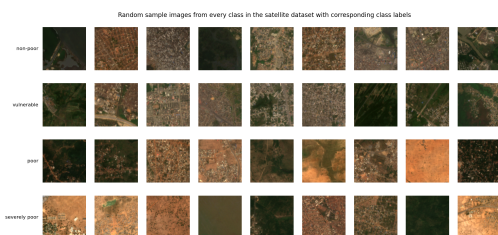


Figure 4: Random Samples of Each Class.

Finally, we looked at some random samples of images from each class (Figure 4) to see whether visual patterns could already be recognized. The non-poor and vulnerable classes often correspond to urban areas with visible housing, structured layouts, and major roads or infrastructure. In contrast, the poor class also shows settlements, but they tend to be less neatly arranged and sometimes less populated, with more barren land.

The severely poor areas are often sparsely populated, rural, and desert-like. Since we only considered urban areas, many of these images appear to be on the outskirts of cities. When settlements are visible in the severely poor class, the housing tends to be less structured compared to the non-poor/vulnerable regions. Overall, these examples suggest that the CNN may be able to pick up structural differences, but it also shows that the task will not be easy, since many images across classes look quite similar and there are cases where an image could plausibly belong to another class at first glance.

This analysis gave us a first impression of the points where we needed to be cautious when building our CNN and the main challenges it was likely to face.

3.2 Methods

3.2.1 Data Preprocessing

Next, we preprocessed the dataset by defining separate transformations for the baseline CNN and ResNet50. The data were split into 70% training, 15% validation, and 15% test sets.

Since the images were slightly larger than 100×100 pixels, we cropped them accordingly for CNN training before creating the dataset archive (see *DLSS_Satellite_Data_Cleaning_Nigeria.ipynb*).

For the **Baseline CNN**, training images were augmented with random resized cropping (scale 0.9–1.1, ratio 0.9–1.1), horizontal flips, small rotations ($\pm 15^\circ$), and slight color jitter (brightness, contrast, saturation, hue). They were then converted to tensors and normalized (mean = std = 0.5 per channel). Validation and test images were only converted and normalized.

For **ResNet50**, we used the standard ImageNet preprocessing: training images were resized to 224×224 , augmented with the same techniques as above, and normalized with ImageNet statistics. Validation and test images were resized and normalized, without augmentation.

Finally, to verify augmentation, we visualized sample images and their augmented versions, which confirmed the intended effects (see *DLSS_Classification.ipynb*).

3.2.2 Model Architecture

We used four model architectures: a **Baseline CNN**, a **Pretrained ResNet50**, and fine-tuned ResNet50 models with **One-Step** and **Two-Step** approaches.

The **Baseline CNN** model is composed of four convolutional layers organized into two convolutional blocks. Each convolutional layer uses a 3×3 kernel with padding and is followed by a ReLU activation function. The first block contains two convolutional layers with 32 filters each, followed by a max-pooling layer to reduce spatial dimensions and a dropout layer with a rate of 0.25 for regularization. The second block contains two convolutional layers with 64 filters each, again followed by max-pooling and a dropout layer with the same rate. After the convolutional feature extraction, the output feature maps are flattened and passed to a fully connected classifier. The classifier consists of a hidden dense layer with 64 neurons activated by ReLU, followed by a dropout layer with a rate of 0.5 to reduce overfitting, and finally an output layer with 4 neurons corresponding to the target classes.

The **Pretrained ResNet50** model, a 50-layer deep residual network, was originally trained on the ImageNet dataset containing over one million natural images across 1000 categories. The choice of ResNet50 is motivated by its strong feature extraction capabilities: features learned from ImageNet generalize well to other domains such as satellite imagery, where recognizing textures, patterns, and shapes is crucial. Using the pretrained weights allows the model to take advantage of transfer learning, which means it can perform well on the classification task with fewer labeled satellite images while still reaching high accuracy. ResNet50 is built using stacked residual blocks with skip connections that address the vanishing gradient problem and enable the training of very deep networks. The architecture begins with a 7×7 convolutional layer and max pooling, followed by four stages of bottleneck residual blocks that progressively increase the number of feature channels from 64 to 2048 while reducing spatial resolution. An adaptive average pooling layer reduces the final feature maps to 1×1 , which are passed into a fully connected layer.

For our task, we replace the final fully connected layer of ResNet50 with a new linear classifier producing four output classes relevant to urban poverty mapping.

With the **One-Step** approach, the Pretrained ResNet50 backbone retains its full architecture, but all convolutional layers are frozen so that the ImageNet features are kept. The only part trained is the new fully connected layer ($2048 \rightarrow 4$), which works as a small classifier for satellite images. The advantage is that training is faster, requires less data, and reduces overfitting, while still benefiting from the strong ImageNet feature representations.

The **Two-Step** fine-tuning model has the same architecture as the One-Step setup, but the training is divided into feature extraction and full fine-tuning. In Stage A, all convolutional and batch normalization layers are frozen, and only the new classification head is trained. This helps the model quickly adjust the output layer to satellite imagery while keeping the general ImageNet features intact. After Stage A, Stage B unfreezes the backbone and continues training with a smaller learning rate. This stage further adapts

deeper features, such as textures, roof types and settlement patterns, making the model more suited to urban poverty mapping.

3.2.3 Model Training

The models were fine-tuned step-by-step, evaluating performance on the validation set after each change in the hyperparameters (or model architecture in the case of the Baseline CNN Model). If a change improved performance, the updated model was used for the next modification; otherwise, the previous model was kept and changed in a different way.

In total, six **Baseline** CNN Models were trained, varying in loss function, learning rate, and model architecture. First, a model with the above-described architecture (see 3.2.2) was defined and trained using the Adam optimizer with a learning rate of 0.001, 100 epochs with early stopping with a patience of 5, CrossEntropyLoss, and L2 regularization (0.0001). To address class imbalance, the second model incorporated class weights in the CrossEntropyLoss function, calculated by inverting and normalizing class counts. In the third model, the learning rate was reduced to 0.0001, then to 0.00005 in the fourth model, and finally to 0.00001 in the fifth model. For the sixth model, a third convolutional block with two convolutional layers (with 128 filters) was added. However, the fifth model achieved the best performance on the validation set and was therefore selected as the final model. The final Baseline CNN Model (with the above-described architecture, see 3.2.2) was trained using the Adam optimizer with a learning rate of 0.00001, 100 epochs with early stopping with a patience of 5, CrossEntropyLoss with class weights, and L2 regularization with a factor of 0.0001.

For training the ResNet50 with the **One-Step** approach, four different fine-tuned versions were trained and tested on the validation set, and in the end we chose the one with the best performance. The first trial used a basic setup with 100 epochs, early stopping with a patience of 5, a learning rate of 0.001, a weight decay of 0.0001, an Adam optimizer, and a CrossEntropyLoss function that did not account for class imbalances. In the second trial, we incorporated the same class weights in the loss function that we had used for the baseline model. This already improved the behavior of the learning curves a lot. To further improve results in terms of accuracy and the confusion matrix, we experimented with smaller learning rates - first 0.00005 and afterwards 0.00001 - which slightly boosted performance on the validation set. The final fine-tuned setup for ResNet50 therefore consisted of 100 epochs, early stopping with a patience of 5, a learning rate of 0.00001, a weight decay of 0.0001, an Adam optimizer, and a CrossEntropyLoss function with class weights to account for the class imbalances.

In total, five ResNet50 Models were fine-tuned with the **Two-Step** approach, differing in loss function and learning rate. The first model was trained with the Adam optimizer, using a learning rate of 0.001 for Stage A and 0.0001 for Stage B, 100 epochs with early stopping (patience of 5), CrossEntropyLoss, and L2 regularization (0.0001). The second model addressed class imbalance by adding class weights to the loss function, calculated by inverting and normalizing class counts. In the third model, the learning rates were reduced to 0.0001 (Stage A) and 0.00001 (Stage B). The fourth model lowered the patience in Stage B to 3 to prevent overfitting. For the fifth model, the learning rates were further reduced to 0.00001 (Stage A) and 0.000005 (Stage B). This model achieved the best performance on the validation set and was selected as the final model. In summary, Stage A of the final fine-tuned ResNet50 (Two-Step approach) Model was trained using the Adam optimizer with a learning rate of 0.00001, 100 epochs with early stopping (patience of 5), CrossEntropyLoss with class weights, and L2 regularization (0.0001), while Stage B used a learning rate of 0.000005 and patience of 3.

All the final set ups are summarized in Table 1.

Model	Learning Rate	Loss Function
Baseline CNN (final)	0.00001	CrossEntropy + class weights
ResNet50 (One-Step)	0.00001	CrossEntropy + class weights
ResNet50 (Two-Step)	A: 0.00001, B: 0.000005	CrossEntropy + class weights

Table 1: Final hyperparameter settings.

Note: All models used Adam optimizer, 100 epochs, patience=5, and L2 regularization = 0.0001.

3.2.4 Learning Curves

The learning curves for all three models provided us with clear insights about the training dynamics as well as their generalization ability.

For the **Baseline** CNN, the training loss consistently drops across each epoch (see Figure 5a), showing that the model is learning from its training data. At first, the validation loss also follows a downward trend during the early epochs, indicating that the model is gaining useful insights that could be used on unseen data. However, after approximately 25 epochs, the validation loss starts to plateau and oscillate slightly while the training loss continues to decrease. This growing space between the two losses might mean that the model is overfitting a little. In other words, the model is getting really good at classifying the specific satellite images it was trained on, but its ability to accurately classify new images might be a bit limited. While we believe this is not a severe case of overfitting, it indicates that the model’s generalization is not as strong as that of deeper, pre-trained models.

In contrast, Figure 5b shows that the ResNet50 fine-tuned in **One-Step** demonstrates a more stable and desirable learning pattern. Both the training loss and the validation loss decrease during the early epochs and remain closely together throughout most of the training process. This suggests that the model is learning well and is also generalizing efficiently. However, toward the end (around epoch 45), the validation loss stops improving while the training loss continues to drop, showing mild overfitting. In this case, the gap between the losses is much smaller than the one seen in the baseline CNN.

The **Two-Step** fine-tuned ResNet50 starts out well in the first stage: both curves decrease together and stay close, similar to the pattern observed in the One-Step approach. However, after the transition to the second stage, the training loss drops sharply while the validation loss spikes upward, suggesting severe overfitting (see Figure 5c).

Overall, the Baseline CNN learns to some extent but overfits quite early and struggles to capture all the details needed. The ResNet50 Two-Step approach, while promising at first, ends up performing poorly because of the sharp overfitting after the second stage. The One-Step ResNet50 performs the best, with only a little overfitting near the end and generally very strong generalization.

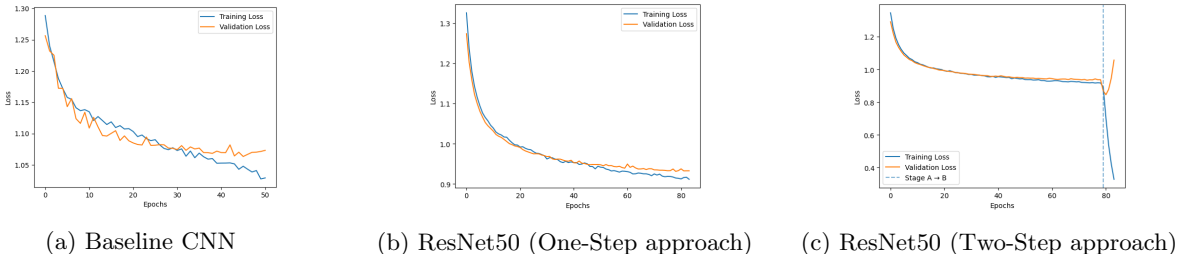


Figure 5: Training and validation loss curves for the three final models.

3.3 Results

3.3.1 Performance Metrics Comparison

The **Baseline** CNN achieved an accuracy of 56.6% which means that the model correctly classified the poverty level for just over half of the satellite images in the test set. The classification report also revealed a critical weakness: the model is highly biased towards the majority class (‘poor’), which it identifies with high precision (0.87) but only moderate recall (0.56), meaning it misses a significant portion of this class. The ‘non-poor’ class has a low precision (0.20), but a surprisingly moderate recall (0.45), indicating it frequently mislabels other classes as ‘non-poor’ but also captures nearly half of the actual ‘non-poor’ images. The very low F1-scores for ‘non-poor’ and ‘vulnerable’ (0.27 for both) confirm the model’s inability to make reliable predictions for these classes. In other words, while the model learns general patterns, it’s unable to discern the nuanced features, thus leading to poor generalization.

The **Pre-trained** ResNet50 without fine-tuning performed exceptionally poor, with an accuracy of only 13.6%. The model’s predictions are severely skewed, showing an extreme bias towards the ‘vulnerable’ class. It has an extremely low precision (0.07), but a very high recall (0.62), meaning it incorrectly labels

the majority of the dataset as ‘vulnerable’ while somehow still capturing most of the actual ‘vulnerable’ instances. The near zero recall for all other classes shows it almost never predicts them. This behaviour is expected and it demonstrates that the pre-trained models are useless for this task without further fine-tuning.

Class	Baseline CNN				Pre-trained ResNet50			
	Precision	Recall	F1-Score	Support	Precision	Recall	F1-Score	Support
Non-poor	0.1955	0.4483	0.2723	116	0.0215	0.0381	0.0275	105
Poor	0.5191	0.6076	0.5599	2016	0.1559	0.0551	0.0814	2016
Severely poor	0.8701	0.5568	0.6790	2924	0.4377	0.1210	0.1895	2902
Vulnerable	0.1944	0.4608	0.2735	408	0.0731	0.6236	0.1308	441
Accuracy			0.5661	5464			0.1356	5464
Macro $\emptyset$	0.4448	0.5184	0.4462	5464	0.1720	0.2094	0.1073	5464
Weighted $\emptyset$	0.6758	0.5661	0.5962	5464	0.2963	0.1356	0.1418	5464

Table 2: Classification results for Baseline CNN and Pre-trained ResNet50 (without fine-tuning).

The fine-tuned ResNet50 (**One-Step** approach) shows a significant improvement, achieving a strong accuracy of 61.3%. More importantly, it demonstrates a far more balanced performance. The precision scores increased for all classes and the progress was even clearer for the minority classes: ‘non-poor’ class rose to 0.18 and the ‘vulnerable’ class rose to 0.27. While still low, this indicates a reduction in the number of false positives for these groups compared to the baseline and the raw pre-trained model. Recall scores also became significantly more even, indicating that the model is sensitive to all classes without severe bias. It now correctly identifies over 50% of the ‘vulnerable’ and ‘non-poor’ instances (0.51, respectively 0.52). The F1-score for the ‘vulnerable’ class doubled compared to the baseline (0.35). This shows that for the minority classes, the model is finding a better balance between making accurate predictions (precision) and capturing a good portion of the actual instances (recall).

The fine-tuned ResNet50 (**Two-Step** approach) achieved the highest accuracy of 67.6% and it also shows the best recall for the ‘non-poor’ minority class (0.47), suggesting a heightened sensitivity to its features. This also contributed to it achieving the highest F1-scores. However, these results might be misleading due to the model’s training. As previously mentioned, the learning curves revealed severe overfitting in the second stage, which means that, unfortunately, the model memorized the training set, including its noise and specific details. Therefore, the model’s performance on truly unseen data would likely be less stable and reliable than the One-Step approach.

Class	ResNet50 (One-Step)				ResNet50 (Two-Step)			
	Precision	Recall	F1-Score	Support	Precision	Recall	F1-Score	Support
Non-poor	0.1752	0.5238	0.2625	105	0.2178	0.4667	0.2970	105
Poor	0.5724	0.6161	0.5934	2016	0.6413	0.6101	0.6253	2016
Severely poor	0.8472	0.6303	0.7228	2902	0.8377	0.7488	0.7908	2902
Vulnerable	0.2716	0.5057	0.3534	441	0.3329	0.5488	0.4144	441
Accuracy			0.6129	5464			0.6761	5464
Macro $\emptyset$	0.4666	0.5690	0.4830	5464	0.5074	0.5936	0.5319	5464
Weighted $\emptyset$	0.6864	0.6129	0.6364	5464	0.7126	0.6761	0.6899	5464

Table 3: Classification results for fine-tuned ResNet50 (One-Step and Two-Step).

It goes without saying that the **One-Step** fine-tuned ResNet50 outperformed all other models. Its strong metrics and stable learning curves indicate that it is a robust and generalizable solution. For this reason, and due to limited space, we decided to focus on this model in the following sections.

3.3.2 Visualization of Prediction Results

To visualize the prediction results of the model, we used a confusion matrix to identify the classes the CNN struggled with. In addition, we displayed sample predictions alongside their true labels to see which structures the CNN was able to recognize and where it had difficulties.

Figure 6a shows that the One-Step ResNet50 correctly predicted 55 non-poor, 1,242 poor, and 1,829 severely poor samples, while the vulnerable class was only correctly predicted 223 times. This means around half of the vulnerable cases were classified correctly, with many being confused as poor (103) or non-poor (90). Overall, the model performs best on the majority class (severely poor) and reasonably well on poor, but struggles significantly with the minority classes, especially vulnerable. The most misclassifications occur between neighboring classes, which is expected given their visual similarities as pointed out in Section 3.1. The learning curves showed controlled overfitting (see Figure 5b), suggesting that the One-Step ResNet50 still provides the best trade-off between accuracy and generalization, even though performance on underrepresented classes remains limited.

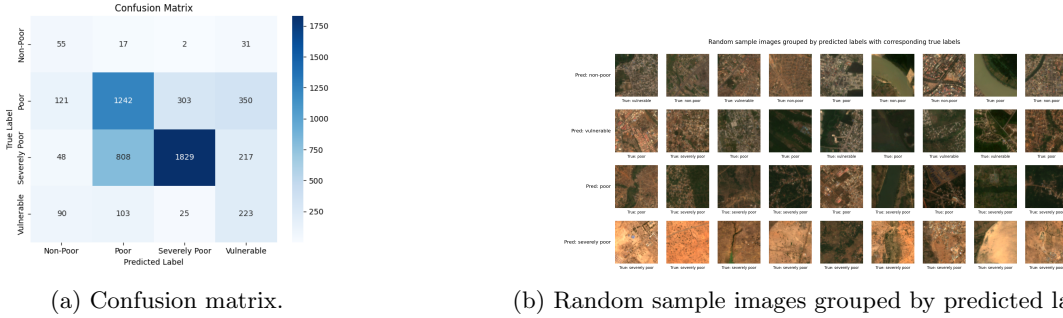


Figure 6: Visualization of Prediction Results.

Furthermore, we can see images predicted as ‘severely poor’ mostly have clear desert-like features with orange tones and little vegetation, and most of these predictions are correct, meaning the model recognizes this class well (see Figure 6b). Images predicted as ‘poor’ usually show areas with sparse buildings and bare ground; many are correct, but some are confused with ‘severely poor’, showing that the model finds it hard to separate these two. Predictions for ‘vulnerable’ often show mixed or semi-urban areas, which explains why the model sometimes mistakes them for ‘poor’ or ‘severely poor’. For ‘non-poor’, most images show dense urban areas with clear roads and infrastructure, but a few are misclassified from ‘vulnerable’ or ‘poor’ classes, likely because some poorer regions still look partly developed.

Overall, the ResNet50 One-Step fine-tuned model works best for severely poor areas, which have strong visual patterns, while differences between the middle classes are harder to detect.

3.3.3 Generalization on Another Country

Testing the model on an entirely new dataset from another country, such as Kenya, is a strong way to evaluate generalizability, because it reveals whether the model’s learned representations capture poverty-related patterns rather than country-specific characteristics. In poverty mapping, regional differences in geography, urban structure, vegetation, and socioeconomic indicators can significantly alter the visual features in satellite imagery, so performance on unseen regions provides a realistic measure of robustness for real-world deployment. When comparing results, the fine-tuned ResNet50 that performed well on Nigeria (accuracy 61.3%, weighted F1-score 0.6364) experienced a major performance drop on Kenya (accuracy 27.9%, weighted F1-score 0.2599). In Nigeria, the model handled the largest classes reasonably well. The ‘severely poor’ class reached an F1-score of 0.7228 and the ‘poor’ class scored 0.5934, while minority classes (‘non-poor’ and ‘vulnerable’) showed lower but acceptable recall. For Kenya, however, the model misclassified heavily across classes, especially ‘poor’, which most samples predicted as ‘severely poor’ or ‘vulnerable’. Although ‘severely poor’ and ‘vulnerable’ achieved relatively higher recall (58% and 46%), their precision was poor (0.2451 and 0.2181), reflecting an overall inability to adapt to new data distributions. This sharp decline highlights that the model’s predictive success in Nigeria relied partly on regional patterns.

Class	Nigeria				Kenya			
	Precision	Recall	F1-Score	Support	Precision	Recall	F1-Score	Support
Non-poor	0.1752	0.5238	0.2625	105	0.6050	0.1477	0.2374	1307
Poor	0.5724	0.6161	0.5934	2016	0.4110	0.1559	0.2260	3689
Severely poor	0.8472	0.6303	0.7228	2902	0.2451	0.5804	0.3447	1175
Vulnerable	0.2716	0.5057	0.3534	441	0.2181	0.4616	0.2962	1497
Accuracy			0.6129	5464			0.2792	7668
Macro $\emptyset$	0.4666	0.5690	0.4830	5464	0.3698	0.3364	0.2761	7668
Weighted $\emptyset$	0.6864	0.6129	0.6364	5464	0.3810	0.2792	0.2599	7668

Table 4: Classification results of the fine-tuned One-Step approach ResNet50 model - Nigeria vs. Kenya.

4 Regression

Even though the classification models produced usable results, their predictive performance was limited, especially for the smaller classes. Since continuous poverty scores were available for the satellite images as well, regression was tested as an additional approach to see if these scores could be predicted and potentially outperform classification. As with classification, four models were trained and fine-tuned: a Baseline CNN, a Pretrained ResNet50, a fine-tuned ResNet50 (One-Step approach), and a fine-tuned ResNet50 (Two-Step approach). Preprocessing was performed in the same way as for classification (see 3.2.1). During fine-tuning, the loss function, learning rate, early stopping patience, and model architecture (for the Baseline CNN) were systematically varied to optimize performance. Due to space limitations, this section only focuses on the fine-tuned ResNet50 (One-Step approach), which offered the best balance between a reasonable learning curve and a strong performance, and was therefore selected as the final model (see *DLSS_Regression.ipynb* for details).

The final regression model is based on the Pretrained ResNet50 architecture, with the original fully connected layer replaced by a single-output layer for predicting continuous values. All other layers were frozen to retain the pretrained features. The model was trained using the Adam optimizer with a learning rate of 0.0001, 100 epochs with early stopping with a patience of 5, MSELoss, and L2 regularization with a factor of 0.0001. During training, the training loss dropped quickly at first and then more gradually, showing steady learning. The validation loss followed a similar downward trend, with occasional fluctuations that sometimes spiked above the training loss. Early stopping occurred at epoch 26, indicating that the model had largely converged (see Figure 7a).



(a) Training and validation loss across epochs.

(b) Predicted vs. true values.

Figure 7: Evaluation of the regression model.

To evaluate model performance, several metrics were used: Mean Absolute Error (MAE), Mean Absolute Percentage Error (MAPE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and the coefficient of determination (R^2). Only values greater than 0.000001 were used when computing MAPE to avoid bias from extremely small values. The model performs reasonably well across most metrics, providing useful predictions (MAE = 0.08, MAPE = 333.87%, MSE = 0.01, RMSE = 0.10, R^2 = 0.60). The high

MAPE looks alarming at first, but this could largely be due to the scale of the values (between 0 and 1) and should therefore be interpreted with caution. The R^2 of 0.60 indicates that the model captures a substantial portion of the variance in the poverty scores, suggesting that it successfully identifies key patterns in the satellite images.

In the predicted vs. true values plot, most data points are spread symmetrically around the diagonal, usually within a deviation of about ± 0.2 poverty score points (see Figure 7b). This further confirms that the model has learned the general patterns in the satellite images quite well. However, it seems to struggle at the edges of the distribution: very low scores (0.0-0.2) are mostly overestimated, and very high ones (0.9-1.0) underestimated, likely because there were fewer training examples in these ranges. Overall, the model captures the trend reasonably well, though the predictions are often not very precise, especially at the extremes.

To further illustrate the results, one random sample image was visualized for each predicted score range from 0.1 to 1.0 (in steps of one tenth), showing both predicted and true values (see Figure 8).

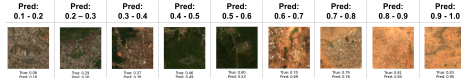


Figure 8: Sample images.

The images reveal how the model was able to capture gradual landscape changes with increasing poverty scores: from large green cities with very high population density, to urban areas with more green spaces and slightly lower density, to forest regions with much lower density, followed by mixed tree-semi-desert landscapes, then desert areas with dense settlements or desert cities, and finally desert regions with much lower population density. This could also help explain why predictions were less precise at the extremes: in the lowest range (0.0-0.2), most images show very similar dense urban green areas, and in the highest range (0.9-1.0), mainly lower-density desert regions, making it difficult for the model to distinguish poverty scores.

Due to space limitations, only a brief note on the model’s generalization capability to Kenya is provided (see *DLSS_Regression.ipynb* for details). Performance is substantially worse than in Nigeria (MAE = 0.13, MAPE = 99.94%, MSE = 0.03, RMSE = 0.16, R^2 = 0.03), with the model mostly predicting values in the mid-range. Predictions are slightly better in high-poverty desert-like areas, but otherwise the landscape seems too different, with less dense urban areas and features not present in Nigeria, such as rocky plateaus. Overall, the model does not generalize well to Kenya.

5 Discussion

Our results show that convolutional neural networks can detect meaningful visual patterns in satellite images that relate to poverty. For the classification task, the Baseline CNN achieved some success (test accuracy around 57% and macro F1-score of 0.45), but it often overfitted and struggled to correctly predict minority classes. Using transfer learning with ResNet50 improved performance noticeably. Among the tested variations, the One-Step fine-tuned ResNet50 provided the best trade-off between accuracy and generalization, reaching a test accuracy of about 61% with a macro F1-score of 0.48. This confirmed that pretrained ImageNet features transfer well to the satellite domain and provide a solid foundation for poverty classification. By comparison, the Two-Step fine-tuning achieved slightly higher accuracy on the training set, but the learning curves and evaluation metrics suggested overfitting, resulting in weaker generalization to new data. However, when evaluated on Kenya, the models showed a notable drop in performance, indicating that generalization across countries is limited and region-specific fine-tuning remains necessary.

Regression was introduced as an additional approach because continuous poverty scores were available in the WorldPop dataset. Unlike classification, which forced regions into four fixed categories, regression produced continuous predictions, allowing for a more nuanced representation of economic variation. The fine-tuned ResNet50 (One-Step) was again the best-performing model, showing steady convergence and explaining a substantial share of the variance in poverty scores (R^2 = 0.60, MAE = 0.08, RMSE = 0.10). The predicted versus true value plots indicated that the model captured general patterns, but it systematically overestimated very low scores and underestimated very high ones, reflecting the scarcity of training data at the extremes. When tested on Kenya, the regression model’s performance dropped sharply (R^2 = 0.03), showing limited transferability across countries with different landscapes and settlement structures.

Comparing classification and regression reveals clear differences. Classification provided interpretable

categories useful for communication but struggled with class imbalance and rigid boundaries between classes. Regression, on the other hand, avoided arbitrary thresholds and better reflected gradual differences between regions, making it more robust and informative when relative variation is more important than discrete groupings.

We also faced several limitations in our project. First, the WorldPop poverty scores were not calculated the same way in every country, which made results harder to compare. Second, there was a time gap between the poverty data and the more recent satellite images we used, which could lower the accuracy of our models. The quality of the images was another issue, some were too dark, patchy, or incomplete, adding noise to the training. When we removed these low-quality images, we also risked leaving out certain regions, which might have biased the results. In addition, we had to create separate data splits for the Baseline CNN and ResNet50, so they were not always trained on the exact same images. Finally, the poverty score itself could be misleading in areas with very few people: if one poor household lived in a sparsely populated area, the poverty percentage could be pushed unrealistically high.

For future work, these challenges could be addressed in several ways. Using high-quality TIFF images instead of compressed JPG/PNG would help preserve more details. Expanding the dataset to include more countries, and even training the model on several countries at once, could make the results more generalizable. Another promising idea is to combine different types of data, for example, daytime satellite images with nighttime light data, to capture multiple aspects of economic activity, as has already been demonstrated in previous work [9].

6 Political Impact Analysis

After showing that – at least to some extent – it is possible to use CNNs for poverty mapping, either through classification or regression, the question remains: How could predictions of such a CNN model inform targeted poverty reduction interventions? And what are the implications and benefits of using satellite images in this way, but also the limitations and ethical considerations that need to be kept in mind?

There are many different ways the model’s predictions could inform poverty reduction. First of all, through poverty mapping: the model can show where interventions are needed by identifying high-poverty regions. Furthermore, CNNs can support landscape analysis – as shown in our results. By identifying landscape features connected to poverty, insights into what poor regions need can be gained. For example, desert regions might require better access to water, while rural villages often need sturdier houses, schools, jobs – in general, better infrastructure.



Figure 9: Example - Access to Water.

Infrastructure differences are another aspect the models can help to identify: large streets and airports are typically found in lower-poverty regions, whereas higher-poverty areas tend to have smaller, less connected roads. Another difference is access to water: rivers and green areas are often linked to lower poverty, while deserts and dry regions are frequently associated with higher poverty scores (see Figure 9).

The model can also distinguish city areas, which usually show lower poverty, from less dense areas that are more likely to have higher poverty. A possible explanation is that access to education, jobs, and health care is worse in less dense areas. Another use case of CNN models in targeted poverty reduction is monitoring: they can compare regions over time and track whether interventions have been successful [8].

So, having a good CNN model can help identify landscape features and infrastructure differences between regions of varying poverty levels, which in turn can inform better-targeted poverty interventions. CNNs can then also help monitor those poverty interventions and their success. To train such a CNN, good-quality satellite imagery is essential, as demonstrated in our project.

Using high-resolution satellite images that capture roofs, settlement density, and farmland supports research, since these features correlate strongly with economic activity, and deep learning makes it possible to analyze patterns at scale. Other studies have shown that CNNs trained on daytime imagery explained up to 75% of the variation in wealth across African countries [9]. The value of using satellite images for research and policy lies in filling gaps in countries with limited survey data, enabling fine-grained poverty maps at regional levels, and supporting monitoring and global development initiatives.

But using satellite imagery also has its limits.

One limitation is that satellites capture only surface features and physical assets (such as roads, roofs, and vegetation) but not the “invisible” yet crucial social and behavioral aspects of poverty, such as income, nutrition, health, education, and consumption [10]. Another limitation is the aggregation mismatch: poverty is a household-level phenomenon, whereas satellite-based predictions are typically made at an area level (pixel, cluster, region). This means that while satellite imagery may be useful for poverty mapping, especially at a regional scale, it may not be appropriate for directly targeting households [9].

Furthermore, satellite-based models raise fairness and ethical concerns, especially since they can exhibit systematic bias. In countries where cities are much wealthier than rural areas (as in Nigeria), models tend to under-allocate aid to rural regions, since they struggle to “see” poverty in rural settings that look visually different from poor urban areas. Conversely, in countries where wealth is less tied to city life (such as the Philippines), the models may over-allocate aid to rural areas. These are not random errors but systematic biases in satellite-based predictions [11]. Using CNNs with such biases risks worsening inequality instead of reducing it. Another concern is privacy: poverty maps can expose or stigmatize communities, and the smaller the defined area, the greater the risk - particularly in terms of data protection. Finally, errors can lead to the exclusion of poor households or the inclusion of non-poor ones, which has serious distributional consequences and undermines trust. And such errors are prone to happen as our results have shown (see 3.3).

All in all, while these models do offer valuable insights and can support better targeting of poverty reduction efforts by enabling poverty mapping in regions where household surveys do not exist, they must be used with care and with full awareness of their limitations. As our results showed, they are strongest for recognizing broad patterns but weaker for small-area or cross-country predictions. In addition, errors and biases can have real social consequences – especially if resources are misallocated, reinforcing inequality, stigmatization, and loss of trust. Therefore, policymakers should treat these models as complementary tools rather than definitive measures. They can complement traditional household surveys, but not fully replace them.

7 Conclusion

The aim of our project was to explore whether convolutional neural networks can be used to map poverty from satellite imagery, offering an alternative to household surveys that are costly, infrequent, and often unavailable at fine geographic scales. We combined WorldPop poverty data for Nigeria with Sentinel-2 images and trained multiple models: a Baseline CNN, a pretrained ResNet50, a One-Step fine-tuned ResNet50, and a Two-Step fine-tuned ResNet50. We first focused on a classification task, where the objective was to assign regions to four predefined poverty categories (non-poor, vulnerable, poor, severely poor), and then tested the models’ ability to generalize to data from Kenya. To capture more nuanced differences in living standards beyond these discrete classes, we additionally explored a regression setup that predicts continuous poverty scores.

The results showed that transfer learning with ResNet50 clearly outperformed the baseline CNN. The One-Step fine-tuned model offered the best trade-off between accuracy and generalization (with an accuracy of about 61.3%), while the Two-Step model achieved higher training accuracy but overfit more strongly. Classification models could detect relevant features such as settlement density and housing patterns but struggled with class imbalance and arbitrary thresholds. Regression models were more stable and captured gradual socioeconomic variations in Nigeria better, though performance declined significantly when applied to Kenya.

From a policy perspective, our findings suggest that satellite-based poverty mapping can be a valuable tool for targeting interventions in regions where survey data are lacking or outdated. At the same time, there are risks: class imbalance, noise in poverty scores, limited generalization across countries, and interpretability challenges mean that the results must be used with caution.

In conclusion, our project shows both the promise and the current limitations of combining deep learning with satellite imagery for poverty analysis. With improvements such as higher-quality data, multi-country training, and multimodal inputs like night-time lights, this approach could become a powerful tool for supporting evidence-based poverty reduction policies.

References

- [1] Statista. Anzahl der Menschen in extremer Armut weltweit in den Jahren 1990 bis 2022 und Prognose bis 2030. Accessed 2025-08-26; 2024. <https://de.statista.com/statistik/daten/studie/1356924/umfrage/globale-armut-anzahl-der-armen-menschen/>.
- [2] Lal BS. The economic and social cost of illiteracy: an overview. *International Journal of Advance Research and Innovative Ideas in Education*. 2015;1(5):663–670.
- [3] Tatem AJ, Gething PW, Bhatt S, Weiss D, Pezzulo C. Pilot high resolution poverty maps. Accessed 2025-08-15; 2013. Accessed 2025-08-27. Dataset.
- [4] Oxford Poverty and Human Development Initiative. Global Multidimensional Poverty Index (MPI. Accessed 2025-08-26); 2025. Accessed 2025-08-26. <https://ophi.org.uk/global-mpi>.
- [5] Wineman A, Alia DY, Anderson CL. Definitions of “rural” and “urban” and understandings of economic transformation: Evidence from Tanzania. *Journal of rural studies*. 2020;79:254–268.
- [6] United Nations Development Programme. Technical Note 5: Multidimensional Poverty Index. In: *Human Development Report 2020: The Next Frontier—Human Development and the Anthropocene*. Accessed 2025-08-26. New York: UNDP; 2020. .
- [7] Gorelick N, Hancher M, Dixon M, Ilyushchenko S, Thau D, Moore R. Google Earth Engine: Planetary-scale geospatial analysis for everyone. Accessed 2025-08-19. *Remote Sensing of Environment*. 2017. Available from: <https://doi.org/10.1016/j.rse.2017.06.031>.
- [8] Nzelibe IU. Satellite and artificial intelligence in mapping multidimensional poverty in Africa. Accessed 2025-08-21. *African Journal on Land Policy and Geospatial Sciences*. 2024;7(1):325–347.
- [9] Jean N, Burke M, Xie M, Davis WMA, Lobell DB, Ermon S. Combining satellite imagery and machine learning to predict poverty. *Science*. 2016;353(6301):790–794.
- [10] Hall O, Dompae F, Wahab I, Dzanku FM. A review of machine learning and satellite imagery for poverty prediction: Implications for development research and applications. *Journal of International Development*. 2023;35(7):1753–1768.
- [11] Aiken E, Rolf E, Blumenstock J. Fairness and Representation in Satellite-Based Poverty Maps: Evidence of Urban-Rural Disparities and Their Impacts on Downstream Policy. In: *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence (IJCAI-23)*. Macao, China: International Joint Conferences on Artificial Intelligence Organization; 2023. p. to appear.